

TRACE 开发计划

属性	内容
文档版本	v2.4
状态	正式发布
适用版本	TRACE V1 — V2.4
编制日期	2026-06-06
最后更新	2026-06-22
编制人	加鹿鸣
审核人	肖弘基
适用范围	TRACE 项目全体开发人员、测试人员及评审方

修订历史

版本	日期	修订内容	修订人
v1.0	2026-06-06	初稿，确立项目范围、V1 闭环目标和四个工作面划分	加鹿鸣
v2.0	2026-06-15	增加 V2.1 评测硬化、V2.2 A/B 收束、里程碑表更新	加鹿鸣
v2.3	2026-06-20	增加 V2.3 可维护性增强、mutation confirmation、runtime binding	加鹿鸣
v2.4	2026-06-22	增加 V2.4 展示与交付、Compose 交付栈、演示 runbook 计划	加鹿鸣

参考文档

文档	用途
<code>docs/需求分析.md</code>	项目功能边界、用例和验收条件
<code>docs/系统设计.md</code>	系统架构、数据模型和接口合同
<code>docs/测试计划.md</code> v2.4	测试策略、分组执行和质量门禁
<code>docs/测试报告.md</code> v1.0	各阶段测试执行结果与缺陷记录
<code>docs/V2实施清单.md</code>	V2 各子任务详细拆解与完成状态
<code>docs/V2.4_演示runbook.md</code>	V2.4 演示操作步骤与验收动作

术语与缩写

术语 / 缩写	说明
TRACE	本项目名称，Test geneRation Agent via Code Execution
Clean Run	在干净（无缺陷）代码版本上执行测试生成与验证的运行
Variant Replay	将已冻结测试集重放到缺陷变体上的确定性执行，不调用 LLM
Contract Guard	在测试生成结果写入前，基于源码证据拒绝弱断言和漂移生成的拦截层
MockLLM	用于后端和前端集成测试的确定性模拟 LLM，无需真实模型接口
Probe Confirmation	通过 clean/buggy 差异验证 auto mutation variant 有效性的机制
Sampled Mutation Score	仅以 confirmed + probe_passed 的变体为分母的 mutation 捕获率指标
runtime profile	记录 executor、镜像、环境安装、测试命令、超时和产物策略的执行合同
A/B 线	V2.2 并行开发中的 A 线（执行/评测平台）与 B 线（项目理解/生成约束）分工

1. 项目概述

TRACE 是一个面向 Python/FastAPI 项目的测试生成智能体平台。系统围绕“生成、执行、追踪、评测”建立完整工程链路：先分析被测项目，再由智能体生成 pytest 测试，随后真实执行测试，读取结构化失败信息，并在受控边界内进行一次反思修复。运行结束后，系统保存执行轨迹、测试结果、报告和评测指标，用于判断生成测试的可运行性、缺陷捕获能力和不同策略之间的效果差异。项目github仓库：<https://github.com/Ao-chii/Trace>

项目建设分为四个工作面。控制面负责项目、快照、计划、运行、报告和实验管理；执行面负责本地执行器、pytest、运行环境、产物和缓存；智能体面负责项目理解、测试规划、测试生成、失败反思和报告生成；评测面负责数据集、种子缺陷、缺陷变体、策略对比和指标统计。各工作面分阶段落地，数据模型、接口定义和指标口径保持统一。

本项目以《需求分析.md》和《系统设计.md》为依据。核心原则包括：测试必须真实执行，智能体轨迹必须可审计，工具边界必须清晰，反思修复不能修改业务代码或削弱断言，评测以种子缺陷捕获率为核心，本地执行器和 Docker 的安全边界必须如实说明。

2. 项目范围

本期交付范围限定在 Python/FastAPI 与 pytest 场景。系统支持本地项目或受控目录作为被测对象，支持 Direct、Plan-and-Execute、ReAct + Reflection 等测试生成策略，支持 MockLLM 与真实大模型接入，支持测试运行过程追踪、报告导出、种子缺陷评测和策略对比展示。

本期不建设通用聊天机器人、通用研发管理系统、任意语言测试平台、自动修改业务源码的编程智能体、模型训练平台或复杂组织权限流。这些能力可作为后续扩展方向，当前版本优先保证测试生成、执行、追踪和评测主线稳定。

本期计划交付范围如下。

范围项	计划内容	交付口径
测试生成闭环	项目分析、测试生成、pytest 执行、一次受约束反思、报告输出	先在 V1 跑通单机闭环，后续扩展到实验评测链路
运行追踪	保存阶段、工具调用、模型输出、pytest 结果、运行事件和产物	形成可审计的 trace、event、artifact、report 结构
策略对比	支持 Direct、Plan-and-Execute、ReAct + Reflection 三类策略	在同一数据集和运行口径下比较指标
评测体系	干净代码生成测试，缺陷变体重放，同一测试集计算捕获率	以 clean replay 和 variant replay 区分误报与捕获
数据集管理	管理 task、seeded bug、variant 和 auto mutation 最小确认链路	先提供可维护的最小 authoring 体验
执行与交付	本地执行器、运行配置、Docker Compose 交付栈	本期不承诺远程 Runner 和分布式调度
前端可视化	项目、运行、轨迹、报告、实验、数据集和指标展示	支撑开发调试、验收演示和结果复盘

3. 开发策略与里程碑

项目采用“先闭环、再评测、再收束交付”的迭代策略。早期优先保证智能体生成的测试能够真实执行并落库，随后把评测从脚本扩展为可复现的数据库实验，再逐步加固项目理解、执行证据、数据集维护和前端演示链路。

阶段划分按风险收敛顺序推进。V1 用于证明端到端闭环可行；V2.1 用于建立可信的评测尺子；V2.2 收束执行平台和项目理解两条主线；V2.3 补齐数据集 authoring、runtime binding、mutation confirmation 和 replay 生命周期；V2.4 面向最终实训展示，统一 UI、报告、Docker Compose 和验收材料。

阶段	时间	计划目标	主要交付物	验收方式
原型与基线	2026-06-06 至 2026-06-11	验证智能体测试生成和评测 harness 的可行性	智能体原型、初始评测脚本、真实模型配置方案	原型演示和关键流程走查
V1 单机闭环	2026-06-12 至 2026-06-14	建立后端数据模型、工具契约、本地执行和前端运行台	14 张核心表、运行状态机、MockLLM 闭环、运行控制台	MockLLM 端到端运行、报告生成、最小评测脚本

阶段	时间	计划目标	主要交付物	验收方式
V2.1 评测 硬化	2026-06-15 至 2026-06-19	将脚本评测升级为可审计实验，接入真实模型和三策略对比	实验数据模型、clean/replay 分层、三策略指标	真实模型 smoke、三策略实验指标、评测异常分类
V2.2 A/B 收束	2026-06-19 至 2026-06-20	收束执行平台和项目理解合同	runtime profile、executor 抽象、source context、Contract Guard、Dataset 最小入口	后端回归、前端构建、A/B 合同走查
V2.3 可维 护性 增强	2026-06-20 至 2026-06-21	推进数据集编辑、mutation operator、runtime binding、replay cache 和 cleanup	Dataset readiness、probe confirmation、mutation operator set、runtime binding manifest	数据集 authoring 流程、mutation confirmation、replay/cache/cleanup 验证
V2.4 展示 与交 付	2026-06-21 至 2026-06-22	收束最终演示链路和软件交付方式	Dataset 到 Experiment 演示链路、LLM option 校验、Compose 交付栈、演示 runbook	UI 彩排、Compose smoke、benchmark suite、最终演示
收尾 优化	2026-06-23	修复合同问题，调整展示细节，补齐项目文档	前端展示优化、合同修复、文档完善	全量回归和文档检查

4. 阶段实施计划

4.1 原型与需求设计阶段

原型阶段重点验证方向和技术可行性。团队先完成一个最小智能体原型，用于验证项目分析、测试生成、pytest 执行和评测脚本之间的基本衔接。同时完成真实模型接入方式调研，明确 provider 配置、异常处理、输出结构校验和 token 限制等基础问题。

本阶段需要把需求边界、核心流程和长期架构明确下来。需求层面要确认 TRACE 的主线是测试生成执行闭环，评测必须能够证明测试是否捕获已知缺陷，反思修复必须受限于生成测试本身。设计层面要完成控制面、执行面、智能体面和评测面的职责划分，并为项目快照、运行配置、策略版本、轨迹、报告和评测数据预留扩展空间。

V3 远程 Runner、对象存储和完整可观测体系在本阶段只做架构预留，不进入近期开发范围。这样可以避免前期投入过多平台化能力，影响核心闭环的落地。

4.2 V1 单机闭环阶段

V1 阶段目标是跑通最小但完整的单机闭环。本阶段计划建设项目、快照、策略版本、测试计划、测试运行、运行任务、生成文件、测试用例、pytest 结果、轨迹、事件、产物和报告等 14 张核心表。Prompt 版本、工具 Schema 版本、完整评测表、远程 Runner 表和复杂权限模型暂不建设。

执行器采用本地 subprocess、独立工作目录、路径白名单和硬超时。该执行方式能支撑开发和演示中的真实 pytest 执行，但不作为强安全沙箱宣传。生成测试默认写入受控目录，工具层必须做路径规范化校验，禁止写业务源码。

V1 的端到端流程为：创建项目，创建项目快照，创建测试计划，触发运行，智能体分析项目并生成测试，系统写入测试文件并执行 pytest，解析用例级结果，必要时进行一次受约束反思，最后保存 trace、event、artifact 和报告。MockLLM 是本阶段的主要解耦手段，用于保证后端、执行器和前端可以先稳定对接。

前端 V1 优先建设项目入口、运行控制台、阶段进度、轨迹时间线、pytest 结果和报告展示。其他页面可以使用较轻的形式承载，不影响主流程验收。

4.3 V2.1 评测硬化阶段

V2.1 阶段把评测能力从固定目录脚本升级为可审计实验。系统应支持在干净代码上生成并验证测试集，将最终测试集冻结，再把同一批测试重放到缺陷变体上。评测指标至少包括种子缺陷捕获率、干净代码误报率、测试集有效性、运行耗时和模型调用成本。

本阶段最重要的口径是区分“确实没有捕获缺陷”和“本次评测不可评价”。provider failure、artifact hash mismatch、setup failure、executor failure、clean replay collection error、上下文不完整和 flaky clean replay 都应进入无效测试集或系统失败分类，不能压成 0 捕获。

三类策略需要在同一数据集、同一目标范围和同一指标口径下比较。Direct 作为基础生成策略，Plan-and-Execute 负责显式拆解测试目标，ReAct + Reflection 负责结合执行反馈进行一次受控修复。真实模型和 MockLLM 的结果分开解释，MockLLM 主要验证系统链路，真实模型用于观察模型输出和策略效果。

4.4 V2.2 A/B 主体收束阶段

V2.2 采用 A/B 双线并行。A 线负责执行平台和评测尺子，B 线负责项目理解、数据集和生成约束。该分工按失败模式划分：A 线保证运行、重放、指标和产物可信；B 线保证模型看到的源码证据、上下文和生成边界可信。

A 线在本阶段将 runtime profile 提升为执行合同，记录 executor、镜像、工作目录、安装命令、测试命令、超时、网络策略、资源限制、产物和清理策略。setup/install 阶段纳入执行流程，replay scheduler、incremental replay cache、runtime evidence、cleanup 和 artifact metadata 进入可审计链路。

B 线在本阶段强化 source context 和 Contract Guard。项目理解从简单 AST 摘要扩展为 target、dependency、support、route、failure、trace 等多类证据；Contract Guard 基于源码证据拒绝弱断言、目标漂移、未知 fixture、2xx-only route oracle、request/response 字段无证据等问题。Prompt 用于引导生成，系统级 Guard 负责兜住不可接受的输出。

本阶段不建设完整 mutation engine、大规模分布式 replay、Prompt/Tool Schema 管理页和多项目 dataset 的完整产品设计。相关能力保留接口和文档口径，放入后续阶段推进。

4.5 V2.3 可维护性与规模化准备阶段

V2.3 阶段将 V2.2 的能力推进到更可维护、更适合演示和后续扩展的状态。A 线继续处理 runtime profile、replay 调度、cache、cleanup、artifact 生命周期和 metrics evidence；B 线继续处理 Dataset authoring、mutation confirmation、Contract Guard retry 和 mutation engine v1 的最小 operator set。

A 线新增 runtime profile 的 `replay_policy`，用于声明 replay 并发、最大重试次数和退避时间。experiment 支持 `runtime_profile_bindings`，多项目数据集可以按 project 或 task 显式绑定运行配置。前端通过 runtime binding manifest 获取绑定关系，避免自行扫描 project/snapshot 并推断执行配置。

B 线补齐 Dataset authoring 的核心闭环，包括 task、seeded bug、variant 的编辑合同，Dataset readiness 后端检查，mutation confirmation 状态展示，以及结构化 retry prompt。mutation engine v1 按小步方式扩展 operator，每一类 operator 都必须具备 discovery evidence、服务端 rediscovery 和 probe confirmation。只有 confirmed 且 probe passed 的 auto mutation variant 才能进入 sampled mutation score 分母。

对于 exception_type、statement_deletion 等更复杂 operator，本阶段先不纳入交付。相关能力需要先补 probe schema 或更强 AST patch 合同，避免制造等价 mutant、无效 mutant 或语法破坏。

4.6 V2.4 展示与交付阶段

V2.4 阶段面向最终实训展示和软件交付。重点收束 Dataset 到 Experiment 的 UI 链路、Experiment draft 原地启动、Metrics/Evidence/Runtime/Cleanup/Report 展示、LLM option 后端合同、Comparison API 模式、Docker Compose 交付栈和最终演示 runbook。

LLM option 由后端统一提供并校验。前端从 `GET /api/v1/llm-options` 获取可选项，创建 experiment 时 `llm_override` 必须匹配 selectable option。Experiment detail 只允许 draft experiment 原地 Start，其他状态展示当前结果或后续操作建议，避免误导用户重复启动同一实验。

Report/Export 提供最小闭环：metrics JSON 下载、run-level report 入口、Dataset 回指和 Runtime execution JSON。Comparison 页面只消费状态为 completed 的真实 API 数据，API 失败时不回退 demo metrics，避免把演示数据和真实评测结果混在一起。

Docker Compose 作为本期推荐交付方式，负责启动 PostgreSQL、Redis、Backend API、Worker 和 Frontend。Compose smoke、三策略 benchmark 和 UI 彩排作为最终交付前的主要验证动作。远程 Runner、分布式调度、对象存储 artifact 和完整 OpenTelemetry 不纳入 V2.4 验收承诺。

5. 工作分解与责任安排

项目采用面向合同的并行开发方式。团队按能力边界划分 A、B、C 三类责任，避免每条策略或每个页面各自形成一套不可对齐的状态机和指标口径。

角色线	责任范围	稳定产出	主要协作对象
A 线：智能体与评测/执行	Agent 策略、工具契约、真实 LLM、评测 harness、runtime profile、replay、metrics、artifact、cleanup	策略运行结果、EvaluationEvent、experiment metrics、runtime/replay evidence	B 线提供 source context 与 dataset；C 线消费 API 和指标
B 线：后端平台与项目理解	数据模型、API、状态机、source context、Contract Guard、Dataset authoring、mutation confirmation	数据库迁移、服务接口、Contract Guard 结果、dataset readiness 和 mutation evidence	A 线消费 dataset 和 guard 结果；C 线消费后端接口
C 线：前端与可视化	项目页、运行控制台、trace timeline、报告、实验、数据集、runtime profile 和 comparison 展示	Vue 页面、API client、指标表格、证据面板、演示链路	A/B 提供稳定 API 和数据口径

A 线维护评测和执行口径，确保一次实验是否可评价、为什么不可评价、相关产物在哪里都能被追溯。B 线维护证据和生成边界，确保模型基于可信上下文生成测试，证据不足时拒绝、降级或标记 incomplete。C 线围绕真实 API 构建可操作的页面，让项目运行、实验对比和交付演示能够被稳定复现。

6. 技术实施计划

后端采用 FastAPI、Pydantic、SQLAlchemy、Alembic、PostgreSQL、Redis 和 Celery。FastAPI 负责 API 合同和参数校验，SQLAlchemy 与 Alembic 负责持久化模型和迁移，Celery 与 Redis 负责异步运行和实验调度。代码组织上，`agents` 处理智能体 workflow，`tools` 封装智能体可调用工具，`services` 实现业务流程，`api/routes` 对外暴露接口，`models` 和 `schemas` 分别承担数据库对象和数据契约。

前端采用 Vue 3、TypeScript 和 Vite。V1 先完成项目列表、运行控制台、trace 和报告展示；V2 后逐步加入实验列表、实验详情、指标表格、capture matrix、EvaluationEvent timeline、Dataset list/detail、runtime profile、cleanup 和 comparison 页面。前端只展示后端已经提供稳定合同的能力，不用页面文案替底层能力背书。

智能体实现坚持轻量自研状态机，第一阶段不引入重型 Agent 框架。核心工具包括项目文件分析、代码读取、源代码检索、测试文件写入和 pytest 执行。所有工具输入输出必须结构化，并进入 trace。模型输出通过 Pydantic 或等价结构校验，Contract Guard 在执行前拦截明显不可信的生成结果。

评测采用 replay-based evaluation。系统先在 clean snapshot 上生成并验证测试集，再冻结测试集，将同一批测试重放到 seeded bug 或 auto mutation variant 上。捕获必须满足干净代码通过、缺陷变体 assertion failure、失败与 ground truth 相关，import、collection、fixture、环境错误或执行器错误不计入捕获。该路线是本项目评估测试有效性的主要技术基础。

7. 质量保证计划

质量保证分为合同测试、单元测试、集成测试、前端构建、smoke、benchmark 和验收材料七类。合同测试用于锁定数据模型、API shape、工具输入输出、Contract Guard、sampled mutation denominator、runtime profile 和 LLM option 的边界；单元测试覆盖服务函数和评测规则；集成测试覆盖 experiment runner、replay scheduler、artifact cleanup、E2E MockLLM 和真实 API 路由；前端通过 TypeScript 检查和生产构建保证页面可交付。

每个阶段收尾时执行对应范围的回归检查。V1 以 MockLLM 端到端和最小评测脚本为主；V2.1 增加实验 runner、clean replay、variant replay 和真实模型 smoke；V2.2 增加 runtime profile、source context、Contract Guard 和 Dataset 最小链路检查；V2.3 增加 mutation confirmation、runtime binding、replay cache 和 cleanup 检查；V2.4 增加 UI 彩排、Docker Compose smoke 和三策略 benchmark。

项目质量门禁如下。

门禁项	执行时机	通过标准
后端目标测试	功能开发和缺陷修复后	相关测试全部通过，warning 已知且不影响结论
后端全量测试	阶段验收和主链路合并前	<code>backend/tests</code> 全量通过，跳过项有明确原因
前端构建	页面能力变更后	<code>npm run build</code> 通过，无类型错误

门禁项	执行时机	通过标准
格式与补丁检查	合并和验收前	<code>git diff --check</code> 通过
API/UI smoke	V2.4 演示前	Dataset -> Experiment -> Metrics / Evidence -> Report 链路可复现
Compose smoke	交付验证时	PostgreSQL、Redis、API、Worker、Frontend 可启动，基础 smoke 通过
Benchmark	最终演示和验收材料	三策略指标可评价，replay 不调用 LLM，结果文件可追溯

8. 配置与变更管理

项目通过 Git 管理源码、文档和配置模板。开发过程采用小步提交和阶段合并，数据库模型变更必须配套 Alembic 迁移和测试。模型变更不能只修改 ORM 或只修改 API schema，必须同时保证迁移、服务层、接口层和测试数据一致。

数据库结构通过 Alembic 管理。迁移文件按日期和阶段命名，例如 V1 基线迁移、V2 运行配置和评测表迁移、V2.2 执行器证据迁移、V2.3 runtime binding 与 replay policy 迁移。涉及数据合同的字段变更，需要同步更新 schema、服务校验、前端类型和相关测试。

运行配置、模型密钥和本地环境文件不进入版本库。`.env`、`.env.local`、`backend/llm.config.json` 等文件仅用于本地或交付环境配置，应被 Git 忽略。LLM config error 响应必须脱敏，不回显 secret、base_url、原始坏配置或本地路径。真实 LLM 与 MockLLM 结果在文档和指标中分开解释。

文档维护以需求、设计、开发计划、运行说明和阶段验收材料为主。需求和设计文档保持长期蓝图，开发计划记录阶段目标和管理安排，运行说明记录交付环境启动方式，验收材料记录阶段检查结果和演示步骤。历史材料统一归档，避免多个版本的事实口径同时流通。

9. 风险管理计划

TRACE 的风险主要来自智能体输出不可控、评测口径漂移、执行边界被误解和演示流程不可复现。项目通过结构化工具、Contract Guard、clean replay、EvaluationEvent、runtime evidence 和文档边界控制这些风险。

风险	影响	控制措施	阶段安排
生成测试能运行但断言无意义	捕获率和报告失真	Contract Guard、人工评分口径、弱 oracle 拒绝、meaningful assertion 规则	V2.1 起纳入评测口径，V2.2 后持续加固
Reflection 为了通过而削弱测试	误把坏测试当作有效测试	只允许修测试自身问题，禁止删核心断言、skip、吞异常和改业务代码	V1 建立边界，V2 持续验证
无效测试集被当成 0 捕获	策略指标被污染	<code>metric_status</code> 、 <code>invalid_test_set</code> 、clean replay、provider/setup/executor 分类	V2.1 作为硬边界

风险	影响	控制措施	阶段安排
replay 重新调用 LLM	缺陷变体可能污染测试生成	replay 只执行冻结测试集，指标检查 <code>llm_calls=0</code>	V2.1 起执行，V2.4 演示复核
本地执行器被误认为安全沙箱	安全边界误导	文档明确 <code>local_subprocess</code> 不是安全沙箱，Docker 也不夸大绝对安全	V1 起保持一致口径
source context 缺证据导致模型编造	生成测试目标漂移或 oracle 错误	source context gate、retrieval trace、Contract Guard、结构化 retry	V2.2/V2.3 加固
auto mutation 分母污染	mutation score 不可信	server-side rediscovery、probe confirmation、confirmed + probe passed 才入分母	V2.3 锁定合同
Docker 或外部环境不可用	演示失败或误计入指标	preflight 展示 unavailable reason，不进入捕获率分母；Compose 单独验证	V2.4 处理
真实 LLM 配置泄露	密钥和本地路径泄露	配置文件忽略入库，错误响应脱敏	全阶段执行

10. 沟通与文档计划

团队沟通以阶段清单、接口合同和验收材料为主。每个阶段需要明确本阶段目标、接口变更、风险边界和验收动作。A/B/C 三条线在进入并行开发前先锁定数据表、工具输入输出、API shape 和 trace 事件结构，避免后续集成时出现口径偏差。

文档维护要求是：需求和设计文档保持长期蓝图，开发计划体现阶段目标和管理安排，运行说明服务于交付部署，验收材料记录真实检查步骤和结论。废弃文档应及时归档，当前事实源保持精简，避免团队在旧口径和新口径之间摇摆。

最终交付文档包括《需求分析》《系统设计》《开发计划》《测试计划》《测试报告》和相关演示材料。

11. 验收与交付计划

项目验收分为功能验收、工程验收、效果验收和交付验收。功能验收关注用户能否完成项目、快照、计划、运行、实验、报告和数据集操作；工程验收关注状态机、数据模型、工具契约、异步任务、产物和日志是否清晰；效果验收关注 clean false positive、seeded bug capture、invalid test set 和成本指标是否可解释；交付验收关注 Docker Compose、runbook、UI 彩排和 benchmark 结果是否可复现。

V2.4 以 Docker Compose 作为推荐交付方式。系统启动 PostgreSQL、Redis、Backend API、Worker 和 Frontend 后，评审者可按 runbook 进入 Dataset list，选择 demo dataset，创建三策略 experiment，启动 draft experiment，等待 completed，查看 Metrics、Evidence、Runtime、Cleanup 和 Report / Export，并下载 metrics JSON。Compose benchmark suite 用于支撑完整评测演示，UI 彩排材料用于支撑演示链路复核。

最终验收不承诺 V3 能力。远程 Runner 注册、心跳、long-poll、分布式调度、MinIO/S3 artifact storage、完整 OpenTelemetry、experiment-level Markdown 报告服务和 failed/cancelled experiment 原地 rerun 均列为后续候选。当前版本的交付价值在于：单机和 Compose 环境下，测试生成、执行、追踪、评测和展示闭环可以稳定运行，并且关键指标口径有明确依据。

12. 后续演进建议

项目后续可以进入 V3 平台化阶段，前提是继续保持当前的证据和评测口径。远程 Runner 应采用主动拉取任务的模型，Runner 不直连数据库，不持有数据库凭证，通过 job lease、签名 payload 和结构化 result callback 保证执行边界。artifact storage 可以从本地文件系统迁移到 MinIO 或 S3，同时保留 content hash 和审计元数据。

智能体能力方面，后续可以扩展更多 Python Web 框架、OpenAPI 辅助生成、更完整的 Prompt/Tool Schema 版本管理和更细粒度的失败归因。mutation engine 可以继续增加 operator，但每一类新增 operator 都应先补 discovery evidence、syntax check、probe schema 和 denominator 合同，不能用数量替代有效性。

产品体验方面，可以补齐 experiment clone、跨数据集对比、指标 projection cache、分页、运行取消传播、Prompt/Runtime/Tool schema 管理页和更完整的报告导出。所有增强都应继续遵守本期形成的基本原则：系统失败不伪装成模型 O 捕获，证据不足不让模型编造，replay 不调用 LLM，本地执行不宣传成安全沙箱。